

CS-31: Mobile Application Development in Android using Kotlin

Unit: Android Components and APIs

5 (A) Answer the following:

1. **GPS stands for:** Global Positioning System
 2. **Full form of API:** Application Programming Interface
 3. **Call ____, which returns a notification object containing your specifications:**
Notification.Builder
 4. **JSON stands for:** JavaScript Object Notation
-

5 (B) Answer in brief (Any one out of two):

1. What is Service?

A **Service** is a component in Android that runs in the background to perform long-running operations **without a user interface**.

It can continue to run **even when the application is not visible**.

Features:

- Runs independently of user interaction.
- Can perform tasks like **playing music, downloading files, or handling network operations**.

Example Use Cases:

- Music player app continues playing music while the user navigates other apps.
- Background data sync.
- Download manager handling large files.

OR

2. How to check the Network Connection?

In Android, the **ConnectivityManager** class is used to check network status.

Example in Kotlin:

```
val cm = getSystemService(Context.CONNECTIVITY_SERVICE) as ConnectivityManager
val activeNetwork = cm.activeNetworkInfo
val isConnected = activeNetwork?.isConnectedOrConnecting == true
```

- `isConnected` returns true if the device has an active internet connection.
- This method helps apps perform network operations **only when the device is connected**.

5 (C) Answer in detail (Any one out of two):

1. Steps to publish an Android application:

Publishing an Android app involves several steps to prepare, sign, and release it on the **Google Play Store**:

- 1. Prepare the app for release:**
 - Remove debug code and logs.
 - Optimize app resources and images.
- 2. Generate a signed APK/AAB:**
 - Use Android Studio's **Build** → **Generate Signed Bundle/APK** option.
 - A **signed APK/AAB** ensures your app is secure and can be published.
- 3. Create a Google Play Developer account:**
 - Sign up at **Google Play Console**.
 - A one-time registration fee is required.
- 4. Fill in app details:**
 - App title, description, category, and content rating.
 - Upload **screenshots**, icons, and promotional graphics.
- 5. Upload APK/AAB to Play Console:**
 - Choose the correct **release track**: internal, closed, or production.
- 6. Set pricing and distribution:**
 - Free or paid app.
 - Select countries and devices supported.
- 7. Submit for review:**
 - Google reviews the app.
 - Once approved, the app is **live on Google Play Store**.

Key Notes:

- Ensure **app permissions** are correctly declared.
- Test thoroughly before release.

OR

2. Short note on Telephony API:

Telephony API allows Android apps to access phone-related services like **calls, SMS, and network information**.

Main Classes:

- TelephonyManager: Access device info, SIM info, network type.
- SmsManager: Send and receive SMS messages programmatically.

Example Uses:

- Detecting SIM card details.
- Making and managing phone calls.
- Sending OTP via SMS.
- Checking mobile network status.

Benefits:

- Helps developers integrate **communication features** directly into apps.
 - Useful for apps that require **call or SMS functionality**.
-

5 (D) Write a note on (Any one out of two):**1. What is Notification? Explain in detail:**

A **Notification** is a message that appears **outside the app's user interface**, usually in the **status bar** or notification panel.

It informs users about events or updates **without opening the app**.

Features:

- Can include **text, icons, images, and action buttons**.
- Appears even when the app is in the background.

Creating Notifications:

- Use **Notification.Builder** to specify content and style.
- Display it using **NotificationManager**.

Types of Notifications:

1. **Simple Notification:** Basic text and icon.
2. **Big Text Style:** Displays longer text.
3. **Inbox Style:** Shows multiple lines/messages.
4. **Progress Notification:** Shows download or upload progress.
5. **Media Notification:** Controls music or video playback.

Example Uses:

- Alert for new messages.
- Completion of downloads.

- Reminders or alarms.
- Social media notifications.

OR

2. Explain doInBackground() in AsyncTask:

doInBackground() is a method of **AsyncTask** that runs **long-running operations** in the **background thread**, avoiding UI blocking.

Key Points:

- Executes heavy tasks like network operations, database queries, or file downloads.
- Runs independently from the UI thread, keeping the app responsive.

Flow in AsyncTask:

1. **onPreExecute():** Prepares the task (runs on UI thread).
2. **doInBackground():** Executes background operations.
3. **onProgressUpdate():** Updates progress on UI thread.
4. **onPostExecute():** Sends results to UI thread after completion.

Example Use:

- Downloading an image from the internet and updating the ImageView after completion.